

Mathematical Theory of Finance Yield Curve

Shmell Creon

Preamble

```
library(matlib)
price_data <- read.csv("MToF Bond Prices.csv")
bond_data <- read.csv("MToF Bond Datas.csv")
bond_dates <- read.csv("MToF Bond Dates.csv")
coupons <- bond_data[[2]]
years_until_maturity <- bond_data[[3]]/365
face_value <- 100
```

Problem 4

a) YTM

Calculating YTM

```
calculate_ytm <- function(price, face_value, coupon_rate, time_to_maturity) {

  coupon_payment <- (coupon_rate * face_value) / 2
  n_periods <- time_to_maturity * 2

  # Create YTM polynomial ( (price - discounted coupons)YTM^n )
  bond_equation <- function(ytm) {

    t <- 1:n_periods
    pv_coupons <- sum(coupon_payment / (1 + ytm/2)^t)
    pv_face <- face_value / (1 + ytm/2)^n_periods
```

```

    return((pv_coupons + pv_face) - price)
  }

  result <- uniroot(bond_equation, interval = c(-0.1, 0.5))$root

  return(result)
}

```

Plotting

```

par(mfrow = c(2,3))

for (day in 1:11) {
  Maturity = years_until_maturity
  Yield = numeric(10)
  for (bond in 1:10) {
    bond_yield <- calculate_ytm(price_data[day,bond], 100, coupons[bond],
                                years_until_maturity[bond]) * 100

    Yield[bond] <- bond_yield
  }

  plot(Maturity, Yield,
       type = "o",          # Lines and points
       col = "blue",
       lwd = 2,             # Line width
       pch = 19,           # Solid circle points
       main = "Yield Curve",
       xlab = "Maturity (Years)",
       ylab = "Yield (%)",
       ylim = c(min(Yield) * 0.9, max(Yield) * 1.1))
  mtext(bond_dates[[day,1]])
  grid()
}

```


All together now!

```
all_yields_matrix <- matrix(nrow = 10, ncol = 11)

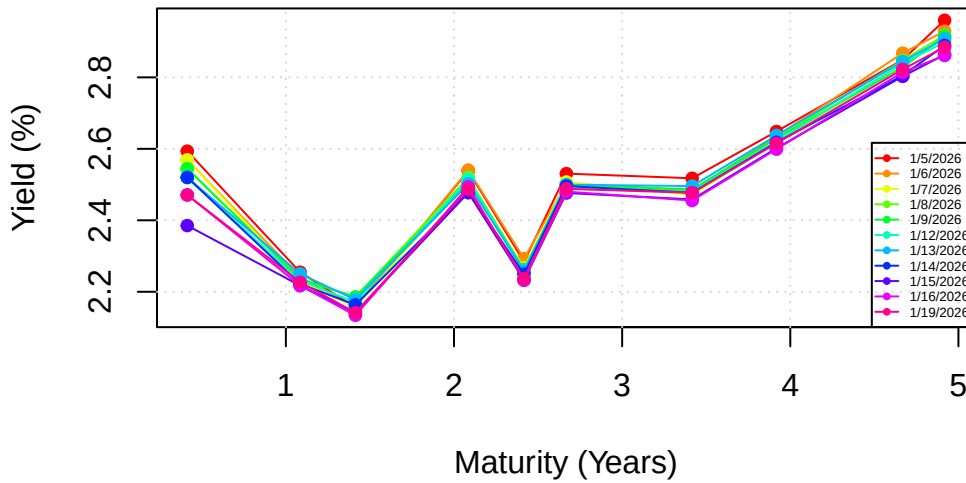
for (day in 1:11) {
  for (bond in 1:10) {
    all_yields_matrix[bond, day] <- calculate_ytm(price_data[day, bond],
                                                  100, coupons[bond],
                                                  years_until_maturity[bond]) * 100
  }
}

plot(years_until_maturity, all_yields_matrix[,1],
     type = "n",
     main = "YTM Curves (All 11 Days)",
     xlab = "Maturity (Years)",
     ylab = "Yield (%)",
     ylim = c(min(all_yields_matrix), max(all_yields_matrix))) # Scale to fit all data
grid()

colors <- rainbow(11)
for (day in 1:11) {
  lines(years_until_maturity, all_yields_matrix[, day],
        type = "o",
        col = colors[day],
        pch = 19,
        cex = 0.8)
}

legend("bottomright", legend = paste(bond_dates[[1]]),
      col = colors, lty = 1, pch = 19, cex=0.4)
```

YTM Curves (All 11 Days)



b) Spot Rates

Pseudo-code

The algorithm for deriving spot rates is reasonably simple, for any day of data our bond maturing in six months is functionally a zero coupon bond, with notional equal to the sum of face value and coupon. We now have our observed price, a notional and time until stock matures, using the compounding formula, $r_1 = \left(\frac{F+C}{P}\right) - 1$, we can find our spot rate for our first bond. For our n'th bond we proceed inductively, assuming we have r_j for $j < n$. Rearranging our price formula, $P = \sum_{t=1}^{n-1} \frac{C}{(1+r_t)^t} + \frac{F+C}{(1+r_n)^n}$, we solve that $r_n = \left(\frac{F+C}{P - \sum_{t=1}^{n-1} \frac{C}{(1+r_t)^t}} - 1\right)^{1/n}$

Making a Spot Rate Function

```
spot_rate.day <- function(x){
  # Take x day's prices
  current_prices <- as.numeric(price_data[x, ])

  # Empty vector to store our spot rates
  spot_rates <- numeric(length(current_prices))
```

```

# Loop through the bonds
for (i in 1:length(current_prices)) {

  # Get data for the current bond "i"
  P <- current_prices[i]
  T <- years_until_maturity[i]
  coupon_payment <- (coupons[i] * face_value) / 2

  if (i == 1) {
    # Base Case
    notional <- face_value + coupon_payment

    # Add spot to our rates
    spot_rates[i] <- 2 * ((notional / P)^(1 / (2 * T)) - 1)
  }

  else {
    # Recursive Case
    pv_prior_coupons <- 0

    # Loop over the bonds who's spot rates we already have
    for (j in 1:(i-1)) {
      t_cashflow <- years_until_maturity[j]
      r_spot <- spot_rates[j]

      # Discount coupon using the spot rate, and add to running turtle
      pv_prior_coupons <- pv_prior_coupons + ( coupon_payment / (1 + r_spot/2)^(2 * t_cashflow) )
    }

    # Calculate dirty price, and use the zero coupon formula.
    dirty_price <- P - pv_prior_coupons

    # The notional is Face Value + Final Coupon
    notional <- face_value + coupon_payment

    # Solve for the spot rate at time T
    spot_rates[i] <- 2 * ((notional / dirty_price)^(1 / (2 * T)) - 1)
  }
}
return(spot_rates)
}

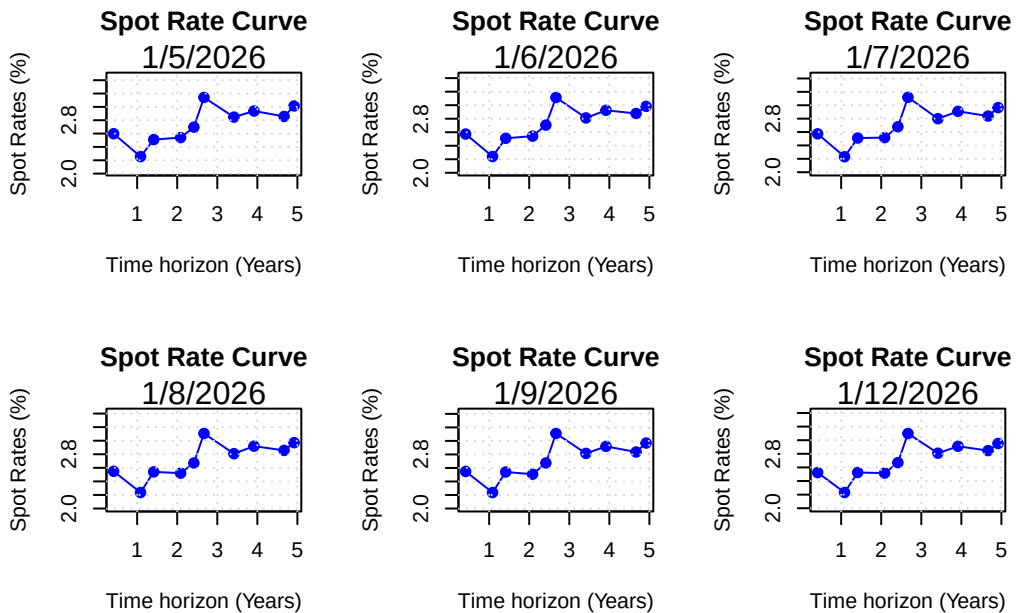
```

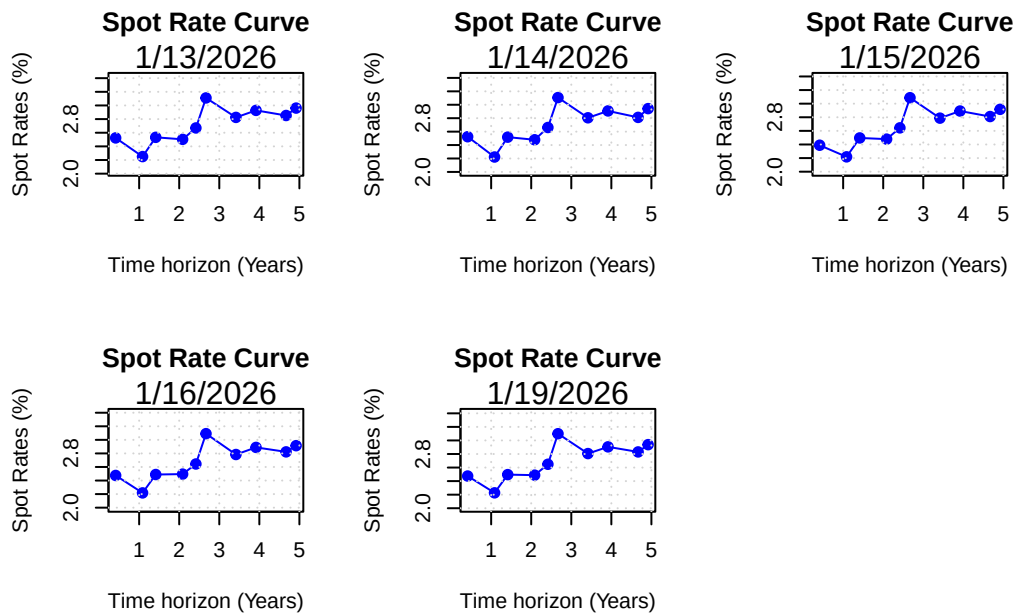
Plotting

```
par(mfrow = c(2,3))

for (i in 1:11) {
  curve_data <- data.frame(
    Maturity <- years_until_maturity,
    Yield <- spot_rate.day(i) * 100
  )

  plot(curve_data$Maturity, curve_data$Yield,
       type = "o",
       col = "blue",
       pch = 19,
       main = "Spot Rate Curve",
       xlab = "Time horizon (Years)",
       ylab = "Spot Rates (%)",
       ylim = c(min(curve_data$Yield) * 0.9,
                 max(curve_data$Yield) * 1.1))
  mtext(bond_dates[[i,1]])
  grid()
}
```





All together now!

```
all_yields <- sapply(1:10, function(x) spot_rate.day(x) * 100)
y_min <- min(all_yields)
y_max <- max(all_yields)

plot(years_until_maturity, all_yields[,1],
     type = "n",
     ylim = c(y_min, y_max),
     main = "Spot Rates for the Next 5 Years (11 Days)",
     xlab = "Time Horizon (Years)",
     ylab = "Spot Rate (%)")

grid()

colors <- rainbow(10)

for (i in 1:10) {
  lines(years_until_maturity, spot_rate.day(i) * 100,
        col = colors[i],
```

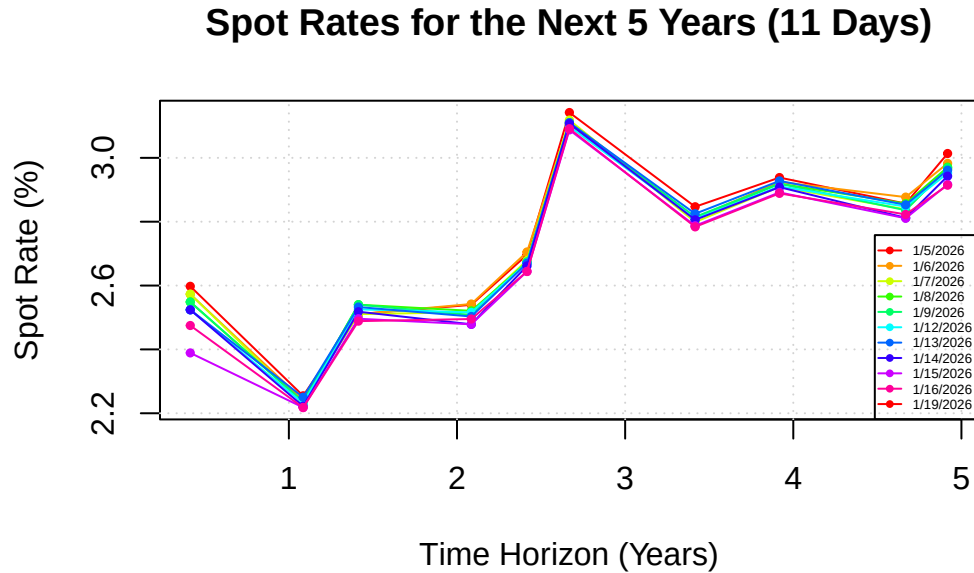


```

    type = "o",
    pch = 19, cex = 0.5)
}

legend("bottomright", legend = paste(bond_dates[[1]]), col = colors, lty = 1, pch = 19, cex=

```



c) Forward Rates

Pseudo Code

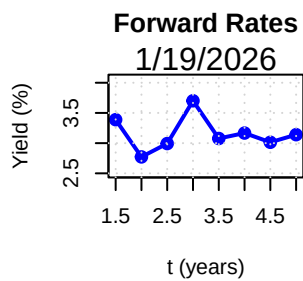
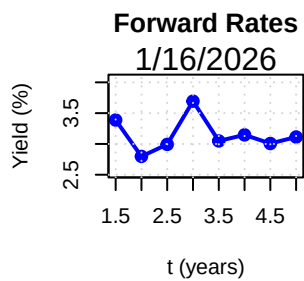
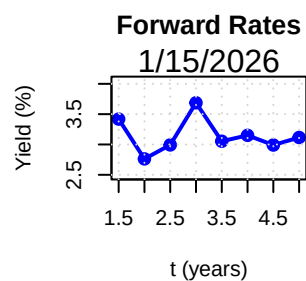
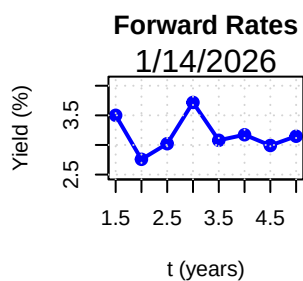
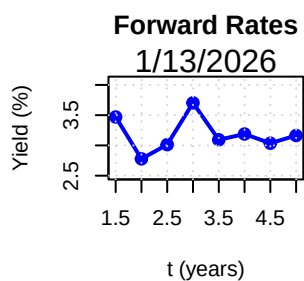
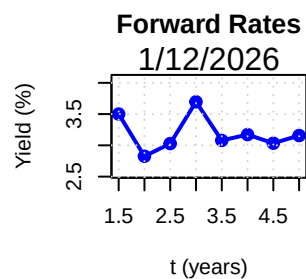
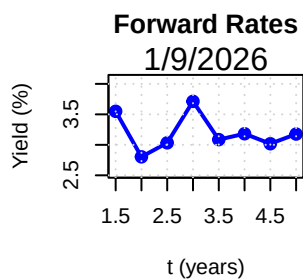
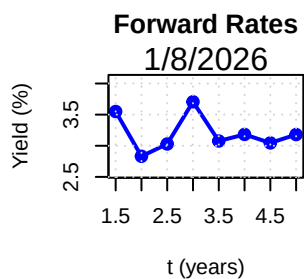
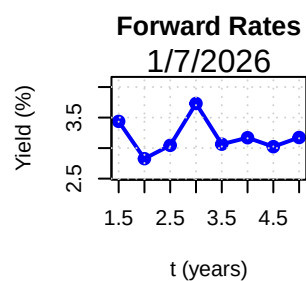
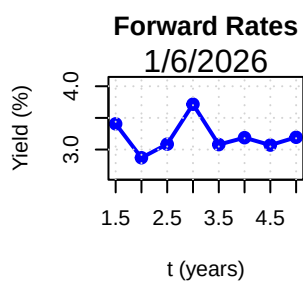
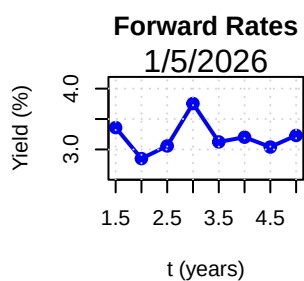
To calculate the forward rate we simply have to use the formula $F_{t,t+n} = \left(\frac{(1+S_{t+n})^{t+n}}{(1+s_t)^t} \right)^{\frac{1}{n}} - 1$. S_t is the spot rate for time t , in this problem $t = 1$. $t + n$ therefore is the difference between the maturity dates of a bond maturing in one year and a bond maturing in n years. Looping over our spot rates for 2 to 5 (although I did 1.5 to 5 in 0.5 increments since we have the data) we get our desired forward rates.

Plotting

```
par(mfrow = c(2,3))
x_axis <- 0.5*c(3:10)
Forward <- numeric(8)

for (day in 1:11) {
  spot_rate <- spot_rate.day(day)

  for (rate in 3:10) {
    n <- (years_until_maturity[rate]-years_until_maturity[2])
    Forward[rate-2] <- (((1+spot_rate[rate])^years_until_maturity[rate]) / ((1 + spot_rate[2])^n))
  }
  Forward <- Forward * 100
  plot(x_axis, Forward,
       type = "o",          # Lines and points
       col = "blue",
       lwd = 2,             # Line width
       pch = 19,           # Solid circle points
       main = "Forward Rates",
       xlab = "t (years)",
       ylab = "Yield (%)",
       ylim = c(min(Forward) * 0.9, max(Forward) * 1.1))
  mtext(bond_dates[[day,1]])
  grid()
}
```



All together now!

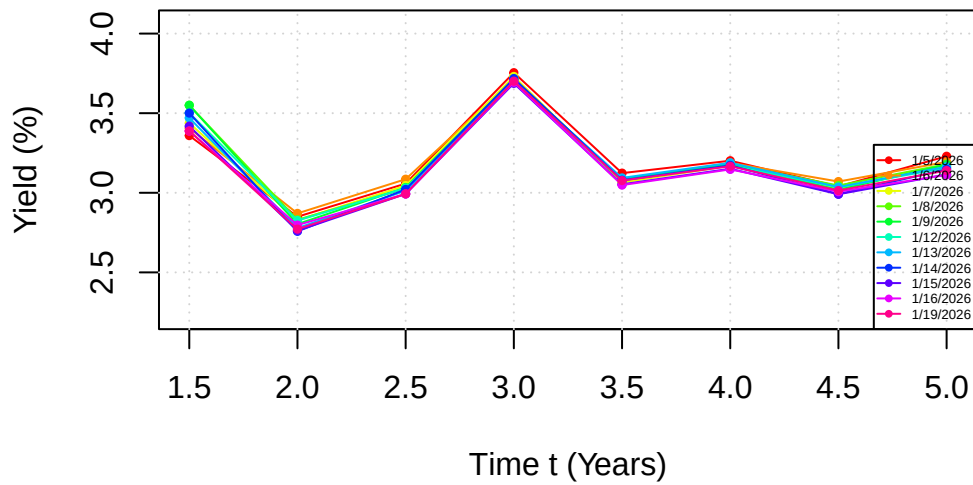
```
plot(x_axis, Forward,
     type = "n",
     main = "Forward Rate Curves (All 11 Days)",
     xlab = "Time t (Years)",
     ylab = "Yield (%)",
     ylim = c(min(Forward) * 0.8, max(Forward) * 1.1))
grid()

colors <- rainbow(11)
for (day in 1:11) {
  spot_rate <- spot_rate.day(day)

  for (rate in 3:10) {
    n <- (years_until_maturity[rate]-years_until_maturity[2])
    Forward[rate-2] <- (((1+spot_rate[rate])^years_until_maturity[rate]) / ((1+spot_rate[2])^n))
  }
  Forward <- Forward*100
  lines(x_axis, Forward,
        col = colors[day],
        type = "o",
        pch = 19, cex = 0.5)
}

legend("bottomright", legend = paste(bond_dates[[1]]),
      col = colors, lty = 1, pch = 19, cex=0.4)
```

Forward Rate Curves (All 11 Days)



Problem 5

YTM Time Series Covariance Matrix

```
YTM_matrix <- matrix(nrow = 11, ncol = 10)
for (bond in 1:10) {
  for (day in 1:11) {
    bond_yield <- calculate_ytm(price_data[day,bond], 100, coupons[bond],years_until_maturity[day,bond])
    YTM_matrix[day, bond] <- bond_yield
  }
}
log_diff_YTM <- diff(log(YTM_matrix))
```

Here is the covariance matrix for YTM, it has been multiplied by 10 to the power of 5 for legibility

```
knitr::kable(cov(log_diff_YTM)*10^5,digits = 3)
```

47.853	1.273	3.806	0.982	3.693	6.591	4.564	3.265	-0.119	5.260
--------	-------	-------	-------	-------	-------	-------	-------	--------	-------

1.273	3.318	2.027	0.861	0.758	1.416	3.321	2.659	2.366	2.616
3.806	2.027	4.525	0.089	0.791	0.756	2.670	2.471	1.740	2.496
0.982	0.861	0.089	3.025	1.032	-1.074	0.009	0.455	3.580	0.096
3.693	0.758	0.791	1.032	2.120	0.018	0.800	1.157	2.082	0.994
6.591	1.416	0.756	-1.074	0.018	2.585	2.621	1.461	-1.217	2.155
4.564	3.321	2.670	0.009	0.800	2.621	4.955	3.344	1.129	4.008
3.265	2.659	2.471	0.455	1.157	1.461	3.344	2.688	1.936	2.874
-0.119	2.366	1.740	3.580	2.082	-1.217	1.129	1.936	5.849	1.205
5.260	2.616	2.496	0.096	0.994	2.155	4.008	2.874	1.205	3.501

Forward Rate TS

```
FR_matrix <- matrix(nrow = 11, ncol = 8)
for (day in 1:11) {
  spot_rate <- spot_rate.day(day)

  for (rate in 3:10) {
    n <- (years_until_maturity[rate]-years_until_maturity[2])
    FR_matrix[day, rate-2] <- (((1+spot_rate[rate])^years_until_maturity[rate]) / ((1 + spot_rate[2])^n))
  }
}
log_diff_FR <- diff(log(FR_matrix* 100))
```

Here is the covariance matrix for time series, it has been multiplied by 10 to the power of 5 for legibility

```
knitr::kable(cov(log_diff_FR)*10^5,digits = 3)
```

25.480	-1.658	1.782	-0.532	0.609	1.600	-0.132	1.683
-1.658	10.615	2.527	-3.441	-3.167	-1.425	6.352	-2.311
1.782	2.527	4.033	-0.610	-0.833	0.213	1.989	-0.085
-0.532	-3.441	-0.610	2.914	1.975	0.584	-3.402	1.658
0.609	-3.167	-0.833	1.975	4.959	2.869	-0.126	4.140
1.600	-1.425	0.213	0.584	2.869	2.207	1.475	2.682
-0.132	6.352	1.989	-3.402	-0.126	1.475	8.350	0.518
1.683	-2.311	-0.085	1.658	4.140	2.682	0.518	3.831

Problem 6

YTM eigenvectors and eigenvalues

```
eigen(cov(log_diff_YTM))
```

eigen() decomposition

\$values

```
[1] 5.143960e-04 1.550739e-04 8.538741e-05 2.773654e-05 1.329417e-05  
[6] 6.030115e-06 1.682848e-06 5.267868e-07 6.002644e-08 1.401688e-20
```

\$vectors

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]
[1,]	0.95394051	0.24515373	-0.11102067	0.013495428	0.05809305	0.032150571
[2,]	0.05858328	-0.41068175	0.06513400	-0.226885966	0.23717833	0.650639494
[3,]	0.10442238	-0.33886003	0.07754796	0.869222911	0.15091662	0.001846177
[4,]	0.02154443	-0.14905659	-0.50828030	-0.232206981	0.35989810	-0.391158173
[5,]	0.08247001	-0.13416951	-0.23581216	-0.016385415	-0.86304222	0.074420031
[6,]	0.14730554	-0.07690028	0.36088644	-0.284880747	0.05111049	0.229186851
[7,]	0.13294488	-0.43168266	0.32155910	-0.221761847	0.01766440	-0.413264130
[8,]	0.09693860	-0.36707588	0.09190477	-0.001769826	-0.15079660	0.086331908
[9,]	0.01713854	-0.40702991	-0.61206834	-0.032106679	0.03954888	0.147474547
[10,]	0.13903929	-0.35144428	0.21621433	-0.085054518	-0.12648814	-0.405675153

	[,7]	[,8]	[,9]	[,10]
[1,]	0.01458121	-0.06006501	0.033221245	0.08976029
[2,]	-0.29250223	0.19710488	0.273953253	0.30519704
[3,]	-0.17627300	0.11395520	0.004442092	-0.21412792
[4,]	-0.41621402	0.35627600	-0.267718303	-0.10333385
[5,]	-0.35241956	0.14462144	0.068135991	-0.13850540
[6,]	0.10157440	0.20572455	-0.235246569	-0.77322155
[7,]	-0.30732715	-0.57574107	0.201434741	-0.07611959
[8,]	0.14399898	-0.12408983	-0.812690177	0.34729880
[9,]	0.50527258	-0.30745498	0.156121965	-0.24669529
[10,]	0.45268155	0.55944697	0.258368602	0.19479392

FR eigenvectors and eigenvalues

```
eigen(cov(log_diff_FR))
```

```
eigen() decomposition
$values
[1] 2.627095e-04 1.938788e-04 1.052205e-04 3.386144e-05 2.311682e-05
[6] 4.154815e-06 8.412807e-07 1.093858e-07

$vectors
      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
[1,] 0.96678093 -0.19601071 0.12108444 0.061220166 -0.07513764 -0.02675072
[2,] -0.16709369 -0.67103496 -0.04274588 -0.129006570 -0.68091525 0.09189822
[3,] 0.04987557 -0.22537278 -0.07230006 -0.871864143 0.38455956 0.16747075
[4,] 0.02770593 0.31165988 -0.03911454 -0.361816278 -0.37374512 -0.72988563
[5,] 0.08922539 0.26534396 -0.54258049 0.002028232 -0.26383254 0.50835001
[6,] 0.09568339 0.07415434 -0.41369776 0.007730773 0.14053767 -0.07835111
[7,] -0.05540542 -0.50521703 -0.50690995 0.277044798 0.35236177 -0.40239903
[8,] 0.11820701 0.17948981 -0.50423901 -0.108321713 -0.17210470 -0.05979984
      [,7]      [,8]
[1,] -0.03585614 0.02929430
[2,] 0.01265356 -0.17626942
[3,] -0.03305807 0.06572523
[4,] -0.29409218 0.10368645
[5,] -0.46620604 0.28610889
[6,] -0.15899688 -0.87348181
[7,] -0.15240966 0.31410504
[8,] 0.80319014 0.09751663
```

Comments

Our first eigenvector has the largest eigenvalue. That means that in this direction we have the most variance of the bonds.

Assumptions Made

It was assumed prices were quoted on the first of January, and that every year has 365 days. Final coupon payment made at the same date of maturity (always). Instantaneous compounding was used to calculate spot rates.